Advanced Algorithms

Unit II

Dr. Ayush Kumar Agrawal

SoCS, UPES

Greedy Paradigm – Foundations

Why Are We Studying Greedy Algorithms?

- Greedy algorithms appear in many classical and advanced optimization problems.
- They are widely used because of their simplicity and efficiency.
- However, greedy algorithms are correct only for problems with special structure.
- In advanced algorithms, our focus is not implementation but correctness.
- This lecture builds the foundation required for matroids, MST, and matching.

What is an Algorithmic Paradigm?

- An algorithmic paradigm is a general strategy for solving problems.
- It represents a way of thinking rather than a single algorithm.
- The same paradigm can be applied to many different problems.
- Examples include Divide and Conquer, Dynamic Programming, and Greedy.
- Understanding the paradigm helps in designing new algorithms.

Intuition Behind the Greedy Paradigm

- The greedy paradigm is based on making decisions step by step.
- At each step, the algorithm selects the option that looks best at that moment.
- The decision is made using only current information.
- Once a decision is made, it is never changed.
- The algorithm hopes that these local decisions lead to a global optimum.

Key Characteristics of Greedy Algorithms

- Greedy algorithms make locally optimal choices at each step.
- Decisions are irreversible; no backtracking is allowed.
- The algorithm progresses in a single forward direction.
- Greedy algorithms are usually fast and memory-efficient.
- Correctness depends entirely on the problem structure.

Local Optimality vs Global Optimality

- A locally optimal choice is the best choice available at a specific step.
- A globally optimal solution is the best possible solution overall.
- Greedy algorithms assume local optimality leads to global optimality.
- This assumption is not universally true.
- Proving this implication is the central challenge of greedy algorithms.

General Structure of a Greedy Algorithm

Typical Greedy Process

- Start with an empty solution set.
- Sort or prioritize all choices using a greedy criterion.
- Consider choices one by one in that order.
- Add a choice if it does not violate feasibility constraints.
- Stop when a complete solution is formed.

Meaning of Feasibility in Greedy Algorithms

- Feasibility ensures that partial solutions remain valid.
- A choice is feasible if it does not break problem constraints.
- Constraints depend on the nature of the problem.
- For graphs, feasibility may mean no cycles are formed.
- For scheduling, feasibility may mean no overlaps occur.

Greedy Choice Property

- A problem satisfies the greedy choice property if a greedy choice is always safe.
- Safe means the choice can be part of an optimal solution.
- At least one optimal solution begins with the greedy choice.
- This property allows the problem to be reduced after each choice.
- Without this property, greedy algorithms cannot be correct.

Optimal Substructure Property

- A problem has optimal substructure if optimal solutions contain optimal sub-solutions.
- After making a greedy choice, the remaining problem must still be optimal.
- This property allows recursive or incremental solution building.
- Optimal substructure is necessary but not sufficient for greedy algorithms.
- Dynamic programming also relies on this property.

Greedy vs Dynamic Programming

- Both greedy and dynamic programming rely on optimal substructure.
- Dynamic programming explores all possible subproblems.
- Greedy explores only one path based on local decisions.
- Greedy is faster but more restrictive.
- Dynamic programming is more general but computationally heavier.

Classic Example: Activity Selection

- The problem involves selecting maximum non-overlapping activities.
- Each activity has a start time and a finish time.
- The greedy strategy selects the activity with the earliest finish time.
- This choice leaves maximum room for remaining activities.
- This problem satisfies the greedy choice property.

Why Greedy Works for Activity Selection

- The earliest finishing activity can always be part of an optimal solution.
- Any optimal solution can be modified to include this activity.
- This modification does not reduce the number of activities selected.
- This reasoning is known as an exchange argument.
- Hence, greedy is provably correct for this problem.

When Greedy Algorithms Fail

- Greedy algorithms fail when early choices restrict future options.
- The algorithm has no mechanism to revise incorrect decisions.
- Local optimality may conflict with global optimality.
- Such problems lack the greedy choice property.
- Understanding failure cases is crucial in advanced algorithms.

Failure Example: Coin Change

- Consider coin denominations $\{1, 3, 4\}$ and target value 6.
- Greedy selects the largest coin first, which is 4.
- The remaining amount is solved using $1 + 1$.
- The greedy solution uses three coins.
- The optimal solution uses only two coins: $3 + 3$.

Why Greedy Fails in Coin Change

- The greedy choice cannot be exchanged later.
- Early selection blocks better combinations.
- There is no structural guarantee for correctness.
- The greedy choice property does not hold.
- This motivates the need for stronger theory.

Exchange Argument: Core Proof Idea

- Exchange arguments are used to prove greedy correctness.
- They show that greedy choices can replace optimal choices safely.
- This replacement does not worsen the solution.
- Exchange arguments rely on structural properties.
- Matroids formalize these properties.

Why This Leads to Matroids

- Greedy algorithms require guaranteed safe exchanges.
- These guarantees are not accidental.
- Matroids capture exactly the structure greedy needs.
- They explain why greedy works for MST.
- The next lecture introduces matroid theory formally.

Summary

- Greedy algorithms build solutions using local decisions.
- Local optimality does not always imply global optimality.
- Greedy correctness depends on special structural properties.
- Exchange arguments justify greedy choices.
- Matroids provide the formal foundation for greedy algorithms.

Matroids – Introduction and Definitions

Why Do We Need Matroids?

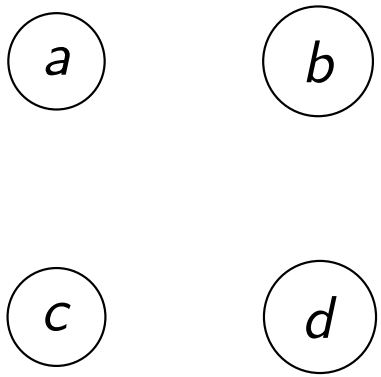
- Greedy algorithms work only for problems with special structure.
- Many different problems admit greedy solutions.
- At first glance, these problems appear unrelated.
- We need a unifying mathematical framework.
- Matroids provide this framework.

What is a Matroid? (Big Picture)

- A matroid is an abstract structure capturing independence.
- The idea comes from linear algebra and graph theory.
- Elements are abstract, not necessarily numbers or edges.
- Independence follows specific rules.
- These rules make greedy algorithms correct.

Ground Set and Independent Sets

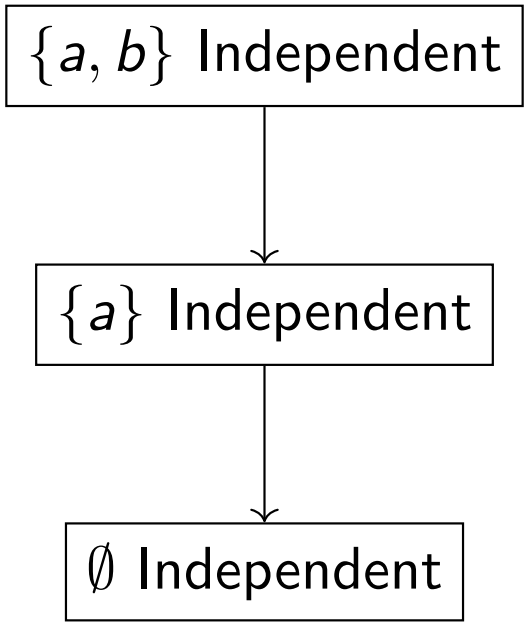
- Every matroid is defined on a finite ground set.
- Some subsets of this ground set are independent.
- Independence is a property we define, not assume.
- The choice of independent sets determines the matroid.



Ground Set $E = \{a, b, c, d\}$

Independence System (Visual Intuition)

- Independent sets must satisfy downward closure.
- If a set is independent, all its subsets are independent.
- This is known as the hereditary property.



Formal Definition of Independence System

Independence System

A pair $(E, \mathcal{I})$ where:

- E is a finite ground set.
- $\mathcal{I}$ is a family of independent subsets.
- If $I \in \mathcal{I}$ and $J \subseteq I$, then $J \in \mathcal{I}$.

Why Independence Systems Are Not Enough

- Independence systems allow different maximal independent sets.
- These maximal sets may have different sizes.
- Greedy algorithms may get stuck early.
- This breaks optimality.
- We need an additional constraint.

Exchange Property (Core Idea)

- The exchange property balances independent sets.
- It prevents one path from being worse than another.
- Smaller sets can always grow.
- This property ensures greedy correctness.

$$\boxed{I_1 = \{a\}} \xleftarrow{\text{exchange element}} \boxed{I_2 = \{a, b, c\}}$$

Formal Definition of a Matroid

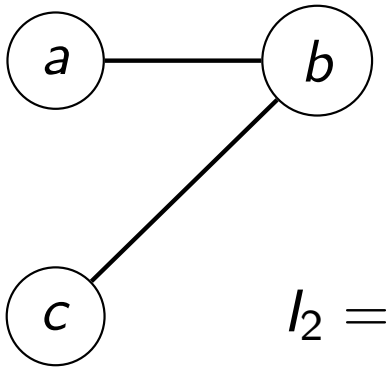
Matroid

A pair $(E, \mathcal{I})$ such that:

- $(E, \mathcal{I})$ is an independence system.
- If $I_1, I_2 \in \mathcal{I}$ and $|I_1| < |I_2|$,
- Then there exists an element in $I_2 \setminus I_1$
- That can be added to I_1 while keeping it independent.

Understanding Exchange Property Visually

- Smaller independent sets are never stuck.
- There is always a way to grow them.
- This avoids bad greedy decisions.

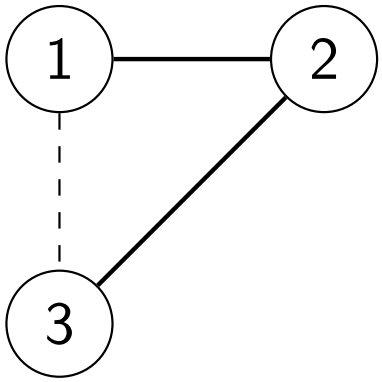


$I_1 = \{a\}$

$I_2 = \{a, b, c\}$

Example: Graphic Matroid (Intuition)

- Consider a graph as the ground set of edges.
- Independent sets are forests (acyclic edge sets).
- Adding an edge that creates a cycle is not allowed.
- This naturally forms a matroid.



Dashed edge creates cycle

Why Matroids Guarantee Greedy Correctness

- Hereditary property ensures feasibility.
- Exchange property ensures fairness.
- All maximal independent sets have equal size.
- Greedy never commits to a bad path.
- This makes greedy provably optimal.

Connection to Upcoming Lectures

- Next, we study greedy algorithms on matroids.
- We compute maximum weight independent sets.
- Then we apply matroids to MST.
- Later, we move to matching and blossoms.

Summary

- Matroids generalize independence.
- They extend independence systems with exchange.
- Exchange property is the key to greedy correctness.
- Many real problems form matroids.
- Matroids unify greedy algorithms.

Greedy Algorithm on Matroids

Where We Are in Unit II

- Lecture 1 introduced the greedy paradigm and its limitations.
- Lecture 2 introduced matroids as the structure greedy needs.
- Now we combine both ideas.
- This lecture shows how greedy works perfectly on matroids.
- This is the theoretical heart of Unit II.

Problem Setting

- We are given a matroid $(E, \mathcal{I})$.
- Each element $e \in E$ has an associated weight $w(e)$.
- Our goal is to select an independent set.
- Among all independent sets, we want maximum total weight.
- This is called the maximum weight independent set problem.

Why This Problem Is Non-Trivial

- There may be many independent sets.
- Trying all independent sets is computationally expensive.
- Local greedy choices may look dangerous in general.
- Matroid structure guarantees safety of greedy choices.
- This is what we will exploit.

Greedy Strategy on a Matroid

- Sort all elements of the ground set by decreasing weight.
- Start with an empty independent set.
- Consider elements one by one in sorted order.
- Add an element if it keeps the set independent.
- Continue until no more elements can be added.

Greedy Algorithm (Formal Description)

Greedy Algorithm on Matroid

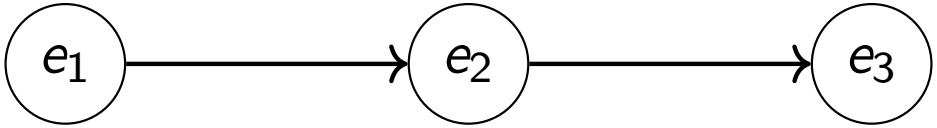
- Input: Matroid $(E, \mathcal{I})$, weight function w.
- Sort E such that $w(e_1) \geq w(e_2) \geq \ldots$.
- Initialize $S = \emptyset$.
- For each e_i in order:
- If $S \cup \{e_i\} \in \mathcal{I}$, then add e_i to S.
- Output S.

What Does This Algorithm Produce?

- The algorithm always produces an independent set.
- The resulting set is maximal with respect to inclusion.
- Among all maximal independent sets, it has maximum weight.
- This is called a maximum weight maximal independent set.
- Matroid properties guarantee optimality.

Visual Intuition: Greedy Growth

- Think of building the solution element by element.
- At each step, we try the best remaining option.
- Independence acts as a safety filter.



Try elements in decreasing weight

Why Greedy Works on Matroids

- Hereditary property ensures feasibility is preserved.
- Exchange property ensures no greedy path is inferior.
- Smaller independent sets can always be extended.
- Greedy never blocks future optimal choices.
- This combination guarantees correctness.

Exchange Argument: High-Level Idea

- Consider greedy solution S and optimal solution O.
- If they are the same, we are done.
- Otherwise, compare elements step by step.
- Exchange property allows replacing elements safely.
- This transforms O into S without loss of weight.

Exchange Argument (Visual)

$$\boxed{\text{Greedy Set } S} \xleftrightarrow{\text{exchange elements}} \boxed{\text{Optimal Set } O}$$

Same size, same weight

Important Consequence of Matroid Property

- All maximal independent sets have the same size.
- Greedy always reaches this size.
- Weight ordering ensures best elements are chosen first.
- Exchange property fixes any disagreement with optimal.
- Hence greedy is globally optimal.

Time Complexity of the Greedy Algorithm

- Sorting elements takes $O(|E| \log |E|)$ time.
- Independence checks depend on matroid type.
- For graphic matroids, cycle checks are efficient.
- Overall complexity is polynomial.
- Practical performance is very good.

What This Means for Algorithm Design

- If a problem forms a matroid, greedy is enough.
- No dynamic programming or backtracking is needed.
- Correctness is guaranteed by structure, not luck.
- This is a rare and powerful situation.
- Many classic algorithms fit this pattern.

Preview: Application to MST

- Graph edges form the ground set.
- Independent sets are forests.
- Kruskal's algorithm is greedy on a matroid.
- Next lecture formalizes this connection.
- This will make MST proofs very intuitive.

Summary

- Greedy algorithm works perfectly on matroids.
- It computes a maximum weight maximal independent set.
- Exchange property is the key to correctness.
- The algorithm is simple and efficient.
- Matroids turn greedy into a guaranteed solution.

Application of Matroids to MST

Why Study MST Through Matroids?

- Minimum Spanning Tree is one of the most important graph problems.
- Classical algorithms like Kruskal and Prim are greedy.
- Earlier, greedy correctness felt problem-specific.
- Matroid theory gives a unified correctness explanation.
- This lecture connects MST directly to matroids.

Recap: What is a Minimum Spanning Tree?

- We are given a connected, weighted, undirected graph.
- A spanning tree connects all vertices without cycles.
- Among all spanning trees, we want minimum total weight.
- Multiple spanning trees may exist.
- MST is the one with minimum cost.

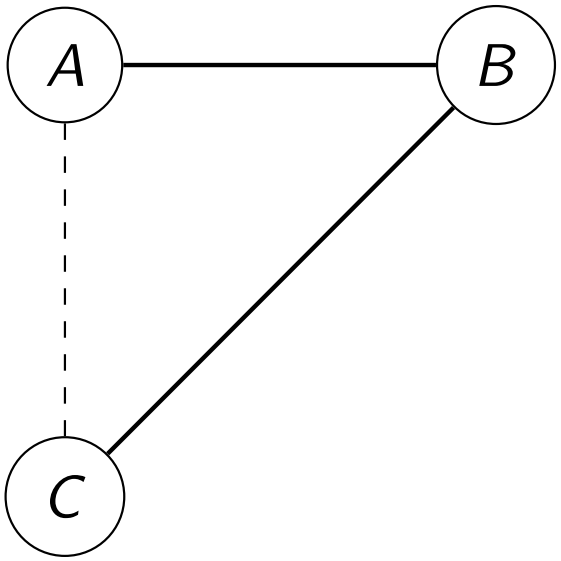
Why Greedy Seems Natural for MST

- We want to pick cheap edges.
- Adding an edge should not form a cycle.
- This matches the greedy mindset perfectly.
- But intuition alone is not a proof.
- Matroids give the missing justification.

Graphic Matroid: The Key Idea

- Consider the graph edges as the ground set.
- Independent sets are edge sets with no cycles.
- Such sets are forests.
- This structure satisfies matroid properties.
- It is called a graphic matroid.

Ground Set and Independent Sets (Visual)



Dashed edge creates a cycle

Edges without cycles form independent sets.

Why This Forms a Matroid

- Empty edge set is independent.
- Any subset of a forest is also a forest.
- If one forest has fewer edges than another:
- An edge can be added without creating a cycle.
- This satisfies the exchange property.

Formal Definition: Graphic Matroid

Graphic Matroid

- Ground set E: edges of the graph.
- Independent sets $\mathcal{I}$: all acyclic edge sets.
- Bases: spanning trees of the graph.

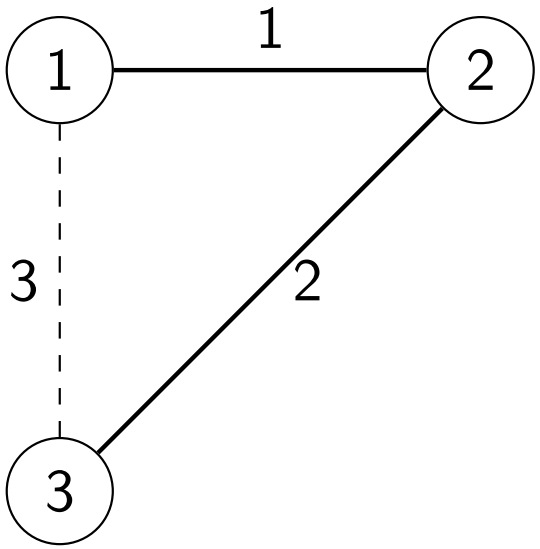
Spanning Trees as Matroid Bases

- A base is a maximal independent set.
- In graphic matroid, bases are spanning trees.
- All bases have the same size.
- Size equals number of vertices minus one.
- This aligns with graph theory results.

Kruskal's Algorithm Revisited

- Sort edges in increasing order of weight.
- Start with an empty edge set.
- Add the next smallest edge if no cycle is formed.
- Stop when a spanning tree is formed.
- This is exactly greedy on a matroid.

Kruskal's Algorithm (Visual)



Skip dashed edge (cycle)

Why Kruskal is Guaranteed Correct

- The graphic matroid guarantees greedy correctness.
- Exchange property prevents bad early choices.
- Lower-weight edges are always safe to try first.
- Independence check prevents cycles.
- Result is a minimum weight base.

Cut and Cycle Intuition

- Every cycle has a maximum weight edge.
- That edge does not belong to the MST.
- Every cut has a minimum weight edge.
- That edge must belong to the MST.
- These intuitions align with matroid theory.

How Matroids Simplify MST Proofs

- No need for complex case analysis.
- MST becomes a special case of greedy on matroids.
- Correctness follows from general theory.
- This abstraction is very powerful.
- Similar ideas apply to other problems.

Time Complexity Considerations

- Sorting edges takes $O(E \log E)$ time.
- Cycle detection can be done using Union-Find.
- Union-Find operations are nearly constant time.
- Overall complexity is efficient.
- Suitable for large graphs.

What We Learned from MST + Matroids

- MST is not just a graph problem.
- It is a matroid optimization problem.
- Greedy correctness is structural.
- Kruskal is greedy on a graphic matroid.
- This insight generalizes well.

Next Topic: Graph Matching

- MST was solvable using greedy + matroids.
- Matching problems are more complex.
- Greedy alone is not sufficient.
- We introduce augmenting paths.
- This leads to Edmonds' Blossom Algorithm.

Summary

- Graphic matroid models acyclic edge sets.
- Spanning trees are bases of the matroid.
- Kruskal's algorithm is greedy on a matroid.
- Matroids give a clean correctness proof.
- MST fits perfectly into greedy framework.

Graph Matching – Introduction and Basics

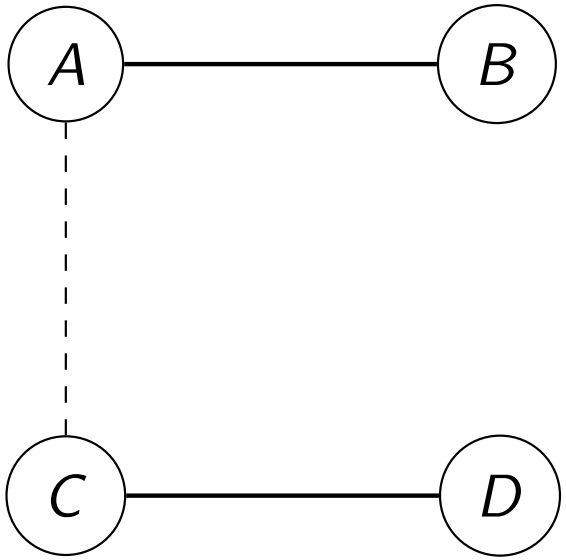
Why Study Graph Matching?

- Matching is a fundamental problem in graph theory.
- It appears in job assignment, resource allocation, networking, and scheduling.
- Unlike MST, matching problems are harder to solve optimally.
- Simple greedy strategies often fail.
- This makes matching a central topic in advanced algorithms.

What is a Matching?

- A matching is a set of edges in a graph.
- No two edges in the matching share a common vertex.
- Each vertex is matched to at most one other vertex.
- Matched vertices are considered occupied.
- Unmatched vertices are considered free.

Matching (Visual Intuition)



Solid edges form a matching

Types of Matchings

- A matching is any valid set of non-adjacent edges.
- A maximal matching cannot be extended by adding another edge.
- A maximum matching has the largest possible number of edges.
- Every maximum matching is maximal.
- Not every maximal matching is maximum.

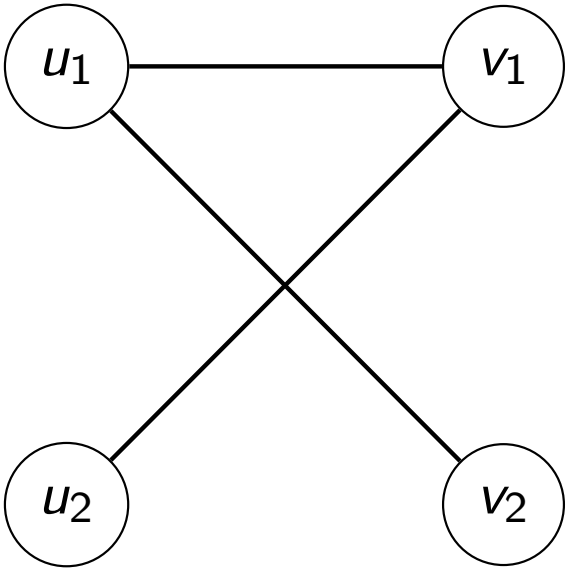
Maximal vs Maximum Matching

- Maximal matching depends on the order of selection.
- Greedy algorithms often produce maximal matchings.
- Maximum matching is globally optimal.
- Finding maximum matching is the real challenge.
- We focus on maximum matching.

Greedy Matching: Why It Fails

- A greedy algorithm picks any available edge.
- Early choices may block better matches later.
- The algorithm has no way to revise decisions.
- This is similar to greedy failures seen earlier.
- Matching lacks matroid structure.

Greedy Failure Example (Visual)



Wrong greedy choice blocks optimal

Why Matching Is Not a Matroid

- Independent sets would be matchings.
- Hereditary property holds.
- Exchange property does not always hold.
- Maximal matchings may have different sizes.
- Therefore, greedy optimality is not guaranteed.

Goal of Matching Algorithms

- Given a graph, find a matching of maximum size.
- Maximize the number of matched vertex pairs.
- Ensure global optimality.
- Allow revision of earlier decisions.
- This requires new algorithmic ideas.

Key Idea: Improving a Matching

- Start with any matching, possibly empty.
- Look for a way to improve it.
- Improvement means increasing matching size by one.
- This leads to the concept of augmenting paths.
- Augmenting paths are the foundation of matching algorithms.

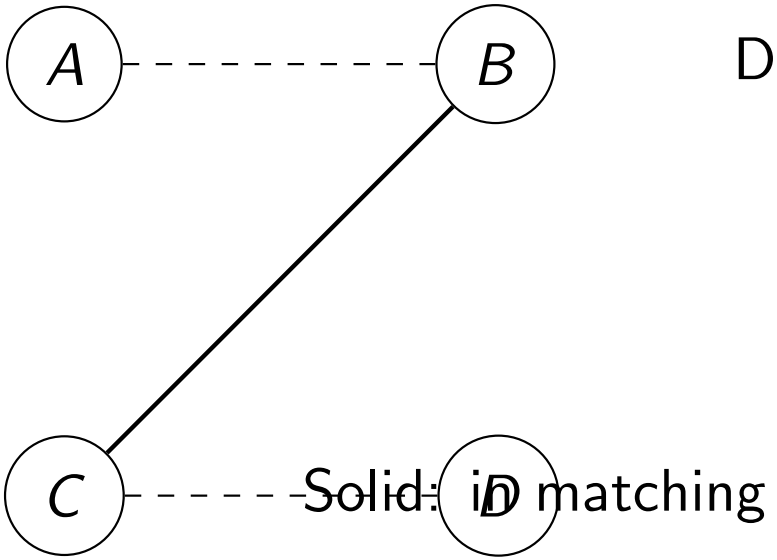
Matched and Free Vertices

- A vertex is matched if it belongs to a matching edge.
- A vertex is free if it is not matched.
- Free vertices are potential endpoints of improvements.
- Augmenting paths always start and end at free vertices.
- Understanding this distinction is crucial.

Alternating Paths (Intuition)

- An alternating path alternates between:
- Edges not in the matching.
- Edges in the matching.
- The path switches between free and matched edges.
- This structure allows controlled modification.
- Augmenting paths are special alternating paths.

Alternating Path (Visual)



Dashed: not in matching

What Comes Next

- We will formally define augmenting paths.
- We will show how they improve matchings.
- We will characterize maximum matchings.
- This leads to polynomial-time algorithms.
- Next lecture focuses entirely on augmenting paths.

Summary

- Matching is a set of non-adjacent edges.
- Maximum matching is the real optimization goal.
- Greedy algorithms fail for matching.
- Matching does not form a matroid.
- Augmenting paths are the key idea.

Augmenting Paths and Maximum Matching

Where We Are in Graph Matching

- In the previous lecture, we introduced matchings.
- We saw why greedy algorithms fail.
- We identified the need to revise earlier decisions.
- Today we introduce the key concept that enables this.
- That concept is the augmenting path.

Goal of This Lecture

- Understand how a matching can be improved.
- Learn the idea of alternating paths.
- Define augmenting paths formally.
- See how augmenting paths increase matching size.
- Characterize maximum matchings.

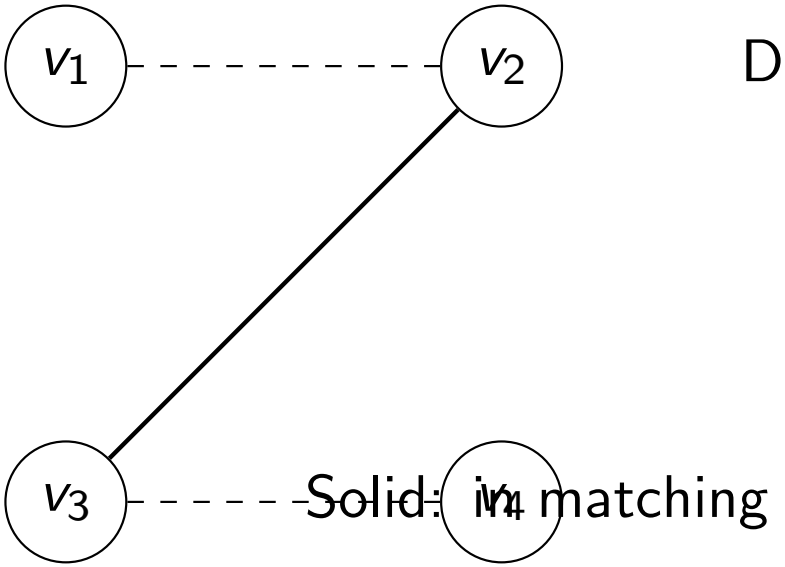
Recap: Matched and Free Vertices

- A vertex is matched if it is part of a matching edge.
- A vertex is free if it is not matched.
- Free vertices are important for improving matchings.
- Any improvement must involve free vertices.
- Augmenting paths always start and end at free vertices.

Alternating Paths: Core Intuition

- An alternating path is a path in the graph.
- Edges on the path alternate between:
- Edges not in the matching.
- Edges that are in the matching.
- This alternating structure allows controlled changes.

Alternating Path (Visual)



Dashed: not in matching

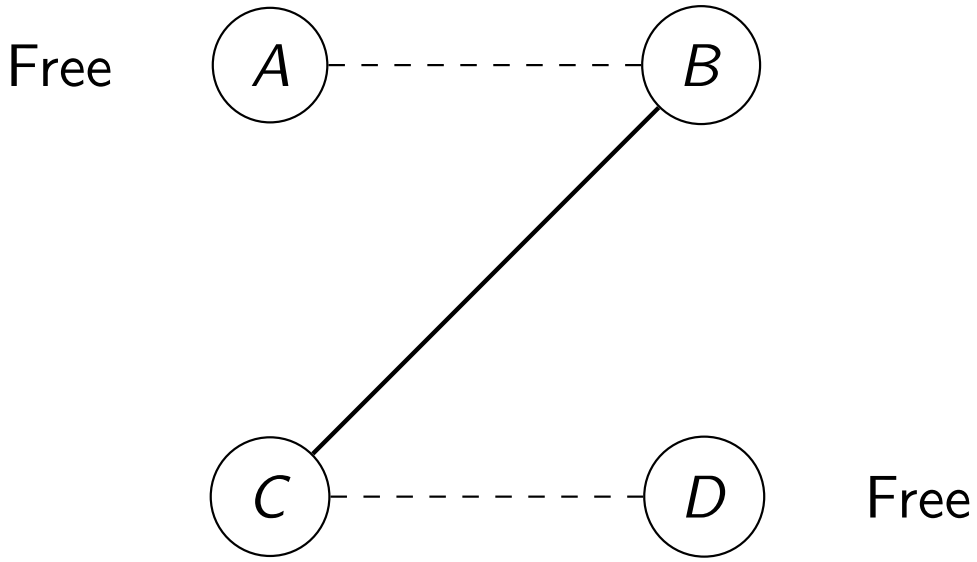
What is an Augmenting Path?

- An augmenting path is a special alternating path.
- It starts at a free vertex.
- It ends at a free vertex.
- The first and last edges are not in the matching.
- The path has odd length.

Why Augmenting Paths Matter

- Augmenting paths allow us to improve a matching.
- They show exactly where greedy went wrong.
- They provide a systematic way to fix mistakes.
- Each augmenting path increases matching size by one.
- This makes them extremely powerful.

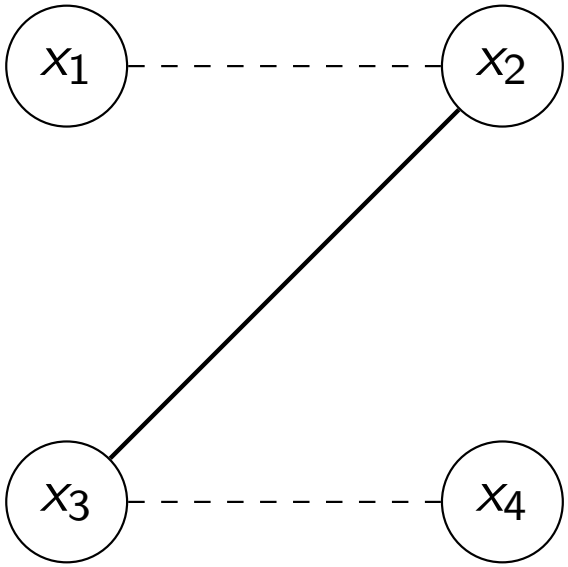
Augmenting Path (Visual Example)



How Augmentation Works

- Take an augmenting path.
- Remove all matching edges on the path.
- Add all non-matching edges on the path.
- The matching remains valid.
- The matching size increases by exactly one.

Augmentation Step (Visual)



Flip edge status

Effect of Augmentation

- Number of matched edges increases.
- No vertex is matched more than once.
- The matching remains valid.
- This is a guaranteed improvement.
- No greedy algorithm can do this.

Key Theorem (Berge's Theorem)

Berge's Theorem

A matching is maximum if and only if there exists no augmenting path with respect to it.

This theorem gives us a precise stopping condition.

Understanding Berge's Theorem

- If an augmenting path exists, matching is not maximum.
- If no augmenting path exists, matching is maximum.
- This is a complete characterization.
- It replaces intuition with certainty.
- All matching algorithms rely on this result.

How Matching Algorithms Use This

- Start with an empty matching.
- Repeatedly search for augmenting paths.
- Augment whenever one is found.
- Stop when no augmenting path exists.
- Result is a maximum matching.

Why This Is Different from Matroids

- Matching does not satisfy matroid exchange.
- But it has augmenting path structure.
- Improvements happen along paths, not single elements.
- This makes matching harder.
- New techniques are required.

Limitations So Far

- Everything discussed works cleanly for bipartite graphs.
- In general graphs, new complications arise.
- Odd cycles create problems.
- Augmenting paths become harder to detect.
- This leads to blossom structures.

What Comes Next

- We move from theory to algorithms.
- We study how to find augmenting paths.
- We encounter odd cycles (blossoms).
- Edmonds' algorithm handles these cases.
- Next lecture introduces blossoms.

Summary

- Alternating paths allow controlled modification.
- Augmenting paths increase matching size.
- Each augmentation improves the matching.
- Berge's theorem characterizes maximum matching.
- This is the foundation of matching algorithms.

Edmonds' Blossom Algorithm – Intuition

Why Do We Need a New Algorithm?

- In Lecture 6, we used augmenting paths to improve matchings.
- That approach works perfectly for bipartite graphs.
- However, general graphs introduce new complications.
- Augmenting paths may exist but are hidden.
- We need a more powerful idea to uncover them.

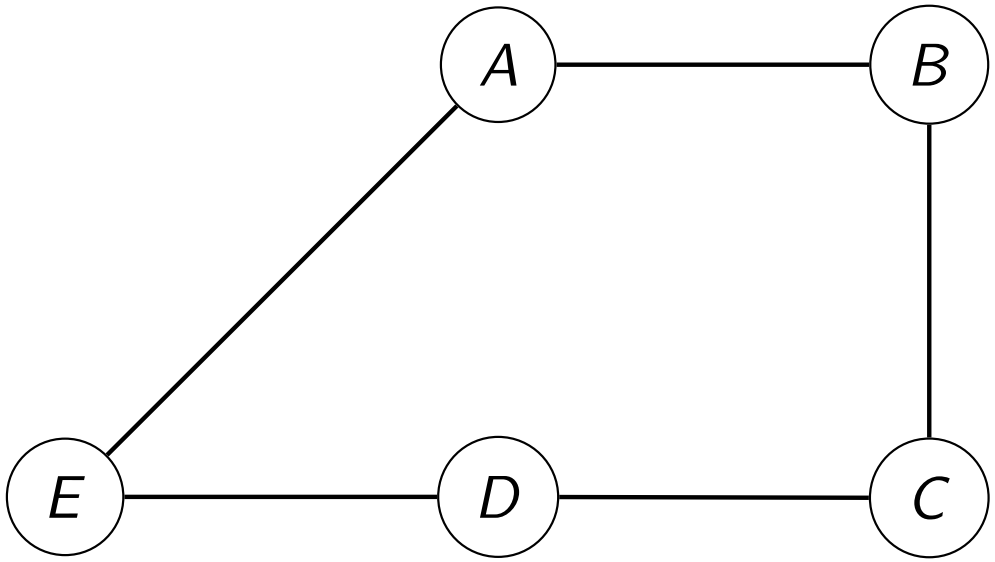
What Goes Wrong in General Graphs?

- General graphs can contain odd-length cycles.
- These cycles interfere with augmenting path discovery.
- A simple BFS or DFS may fail to find augmenting paths.
- The matching may look maximum, but it is not.
- This problem does not appear in bipartite graphs.

Odd Cycles: The Core Difficulty

- An odd cycle is a cycle with an odd number of vertices.
- Odd cycles block alternating path extensions.
- They trap free vertices in a loop.
- This prevents discovering augmenting paths.
- These cycles are called blossoms.

Odd Cycle (Visual)



Odd-length cycle

Why Odd Cycles Are a Problem

- Alternating paths rely on parity.
- Odd cycles break parity consistency.
- We cannot decide which edges to flip.
- Augmenting paths appear blocked.
- But the matching may still be improvable.

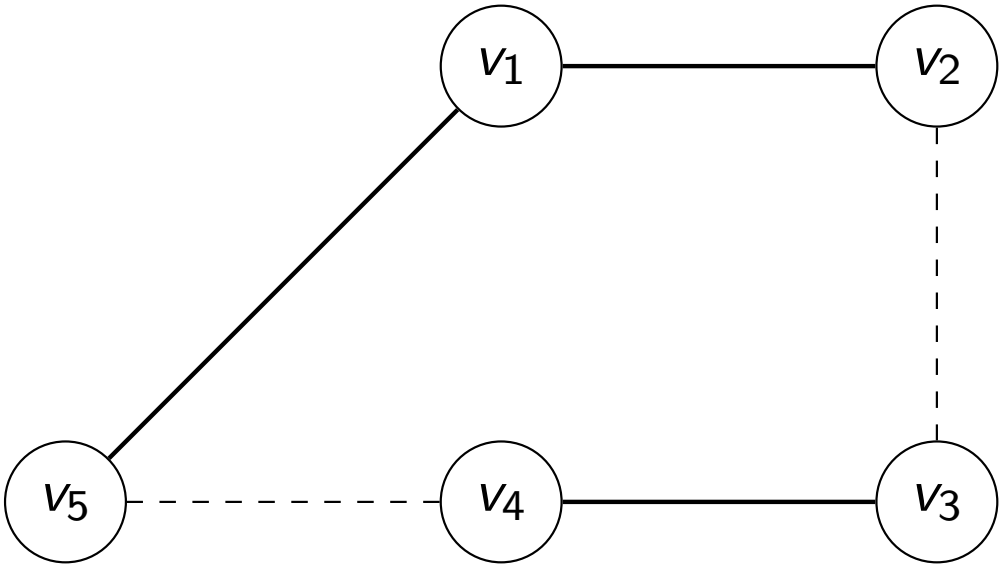
A Failing Example (Intuition)

- Consider a matching inside an odd cycle.
- All vertices appear matched or blocked.
- No augmenting path is visible.
- But a better matching exists.
- Greedy and simple augmentation both fail.

Blossom Structure (Key Insight)

- An odd cycle with alternating matched edges is called a blossom.
- Inside a blossom, augmenting paths get trapped.
- The cycle behaves like a single unit.
- If treated correctly, paths can be revealed.
- This is the core insight of Edmonds.

Blossom (Visual with Matching)

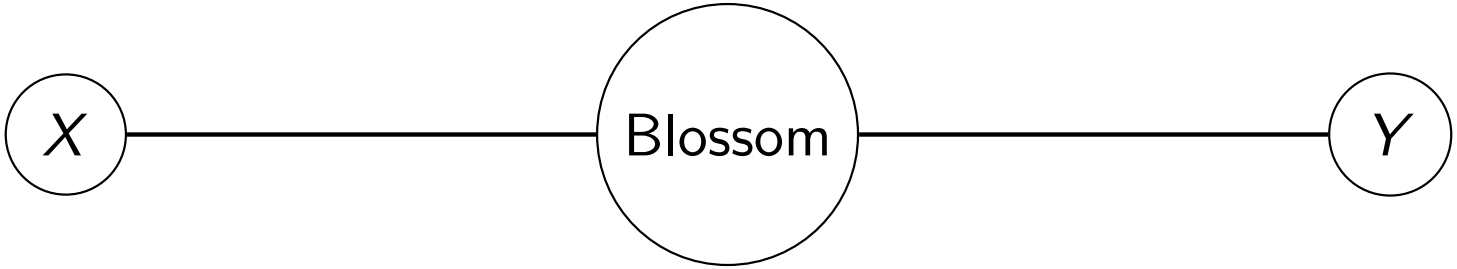


Alternating odd cycle

Key Idea: Shrinking the Blossom

- Treat the entire blossom as a single super-vertex.
- Temporarily contract the odd cycle.
- Search for augmenting paths in the reduced graph.
- This simplifies the structure.
- Augmenting paths may now appear.

Blossom Shrinking (Visual)



Odd cycle contracted

Why Shrinking Works

- Any augmenting path entering the blossom must leave it.
- Inside details are temporarily irrelevant.
- The blossom acts like a single vertex.
- Parity issues disappear after contraction.
- This reveals hidden augmenting paths.

After Finding an Augmenting Path

- Find an augmenting path in the contracted graph.
- Expand the blossom back to original form.
- Carefully reconstruct the augmenting path.
- Perform augmentation in the original graph.
- Matching size increases by one.

What Edmonds' Algorithm Achieves

- It finds augmenting paths in any graph.
- It handles odd cycles systematically.
- It generalizes bipartite matching algorithms.
- It guarantees maximum matching.
- This was a major breakthrough in algorithms.

Why Blossom Algorithm Is Hard

- Blossom detection is non-trivial.
- Shrinking and expanding must be done carefully.
- Implementation is complex.
- Conceptually, the idea is elegant.
- Understanding intuition is more important than code.

What Comes Next

- Next lecture presents Edmonds' algorithm step by step.
- We will see how blossoms are detected.
- We will understand augmentation after expansion.
- Focus will be on algorithmic flow.
- This completes the matching topic.

Summary

- Augmenting paths fail directly in general graphs.
- Odd cycles cause the failure.
- Such cycles are called blossoms.
- Shrinking blossoms reveals hidden augmenting paths.
- This idea leads to Edmonds' algorithm.

Edmonds' Blossom Algorithm – Steps and Working

Recap: What Problem Are We Solving?

- We want to compute a maximum matching in a general graph.
- Greedy algorithms fail for matching.
- Augmenting paths solve the problem in bipartite graphs.
- Odd cycles block augmenting paths in general graphs.
- Edmonds' algorithm overcomes this difficulty.

What Edmonds' Algorithm Does

- Systematically searches for augmenting paths.
- Detects odd cycles during the search.
- Shrinks these odd cycles into single vertices.
- Continues searching in the reduced graph.
- Guarantees finding a maximum matching.

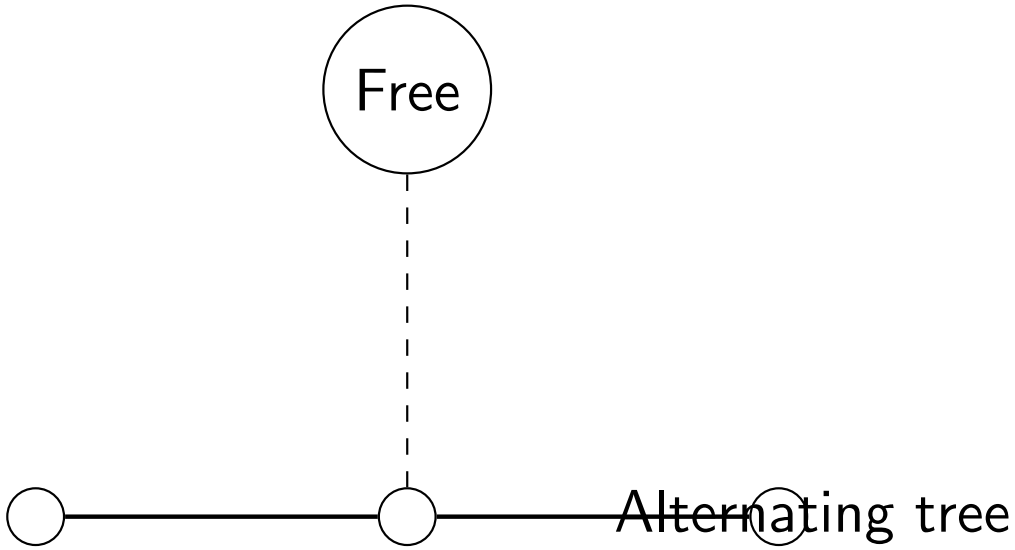
High-Level Structure of the Algorithm

- Start with an empty matching.
- Repeatedly look for an augmenting path.
- If found, augment the matching.
- If blocked by an odd cycle, shrink it.
- Stop when no augmenting path exists.

Search Tree for Augmenting Paths

- The algorithm builds an alternating search tree.
- Rooted at a free vertex.
- Levels alternate between unmatched and matched edges.
- Similar to BFS in bipartite matching.
- Complications arise due to odd cycles.

Alternating Search Tree (Visual)



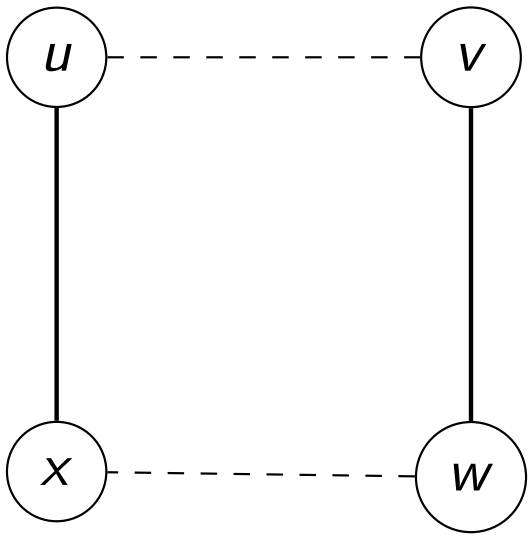
Case 1: Augmenting Path Found

- During search, a free vertex is reached.
- The path from root to this vertex is alternating.
- This path is an augmenting path.
- Perform augmentation by flipping edge status.
- Matching size increases by one.

Case 2: Encountering an Odd Cycle

- During search, two vertices at the same level are connected.
- This forms an odd-length alternating cycle.
- Such a cycle is identified as a blossom.
- Augmenting path search is blocked.
- Blossom shrinking is triggered.

Detecting a Blossom (Visual)

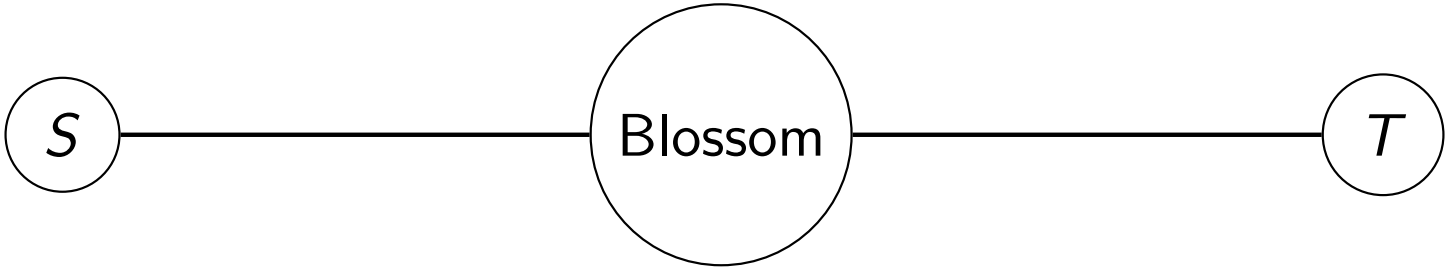


Odd alternating cycle

Blossom Shrinking Step

- Identify all vertices in the blossom.
- Contract them into a single super-vertex.
- Preserve connections to the rest of the graph.
- Maintain alternating structure.
- Continue augmenting path search.

Shrinking a Blossom (Visual)



Contracted odd cycle

Continuing the Search

- Search proceeds on the contracted graph.
- If an augmenting path is found, it may pass through the blossom.
- From outside, the blossom behaves like a single vertex.
- This simplifies parity issues.
- Hidden augmenting paths become visible.

Expanding the Blossom

- Once an augmenting path is found.
- Expand the contracted blossom.
- Reconstruct the path inside the blossom.
- Ensure alternation is preserved.
- Apply augmentation in the original graph.

Why Expansion Is Safe

- Blossom structure guarantees consistent parity.
- There exists a valid internal alternating path.
- Expansion does not break matching validity.
- Augmentation still increases matching size.
- Correctness is preserved.

Algorithm Termination

- Algorithm stops when no augmenting path exists.
- By Berge's theorem, matching is maximum.
- All blossoms have been handled.
- No further improvement is possible.
- Result is a maximum matching.

Time Complexity (High-Level)

- Edmonds' original algorithm runs in polynomial time.
- Complexity is higher than bipartite matching.
- Modern implementations achieve $O(V^3)$.
- Practical performance is acceptable.
- Theory is more important than constants here.

Why Edmonds' Algorithm Is Important

- First polynomial-time algorithm for general matching.
- Introduced blossom shrinking concept.
- Influenced many later algorithms.
- Considered a milestone in algorithm design.
- Central topic in advanced algorithms courses.

Big Picture: Matching vs Matroids

- Matroids allow greedy optimization.
- Matching does not form a matroid.
- Augmenting paths replace exchange property.
- Blossoms resolve parity conflicts.
- Different structures require different tools.

What You Should Remember

- Maximum matching relies on augmenting paths.
- Odd cycles hide augmenting paths.
- Blossoms represent these odd cycles.
- Shrink, search, expand is the core idea.
- Edmonds' algorithm guarantees optimality.

Summary

- Edmonds' algorithm finds maximum matching in general graphs.
- It detects and shrinks blossoms.
- Augmenting paths are found in reduced graphs.
- Expansion restores solutions in original graph.
- This completes the matching part of Unit II.

Summary, Connections, and Exam Perspective

What We Set Out to Learn in Unit II

- Understand when greedy algorithms work.
- Identify the structure required for greedy correctness.
- Apply this understanding to classical problems.
- Handle cases where greedy completely fails.
- Learn advanced techniques beyond greedy.

Unit II at a Glance

- Lecture 1: Greedy paradigm and its limitations.
- Lecture 2: Matroids as a structural guarantee.
- Lecture 3: Greedy algorithm on matroids.
- Lecture 4: Minimum Spanning Tree via matroids.
- Lecture 5–8: Graph matching and Edmonds' algorithm.

Greedy Paradigm: Key Takeaways

- Greedy algorithms make locally optimal decisions.
- Decisions are irreversible.
- Correctness depends on problem structure.
- Greedy is fast but fragile.
- Proof is more important than intuition.

Matroids: Why They Matter

- Matroids formalize independence and exchange.
- They guarantee greedy correctness.
- All maximal independent sets have equal size.
- Greedy never gets trapped in bad choices.
- Matroids explain why MST is solvable greedily.

Greedy on Matroids

- Elements are sorted by weight.
- Feasible elements are added greedily.
- Exchange property fixes disagreements with optimal.
- Result is a maximum weight independent set.
- This is a rare and powerful guarantee.

MST Through the Matroid Lens

- Edges form the ground set.
- Forests are independent sets.
- Spanning trees are matroid bases.
- Kruskal's algorithm is greedy on a matroid.
- Correctness follows directly from matroid theory.

Why Matching Is Different

- Matching does not satisfy matroid exchange.
- Maximal matchings can be suboptimal.
- Greedy algorithms fail badly.
- New tools are required.
- Augmenting paths replace greedy exchange.

Augmenting Paths: Core Insight

- Augmenting paths improve matchings step by step.
- Each augmentation increases matching size by one.
- Berge's theorem characterizes optimality.
- No augmenting path means maximum matching.
- This replaces greedy correctness.

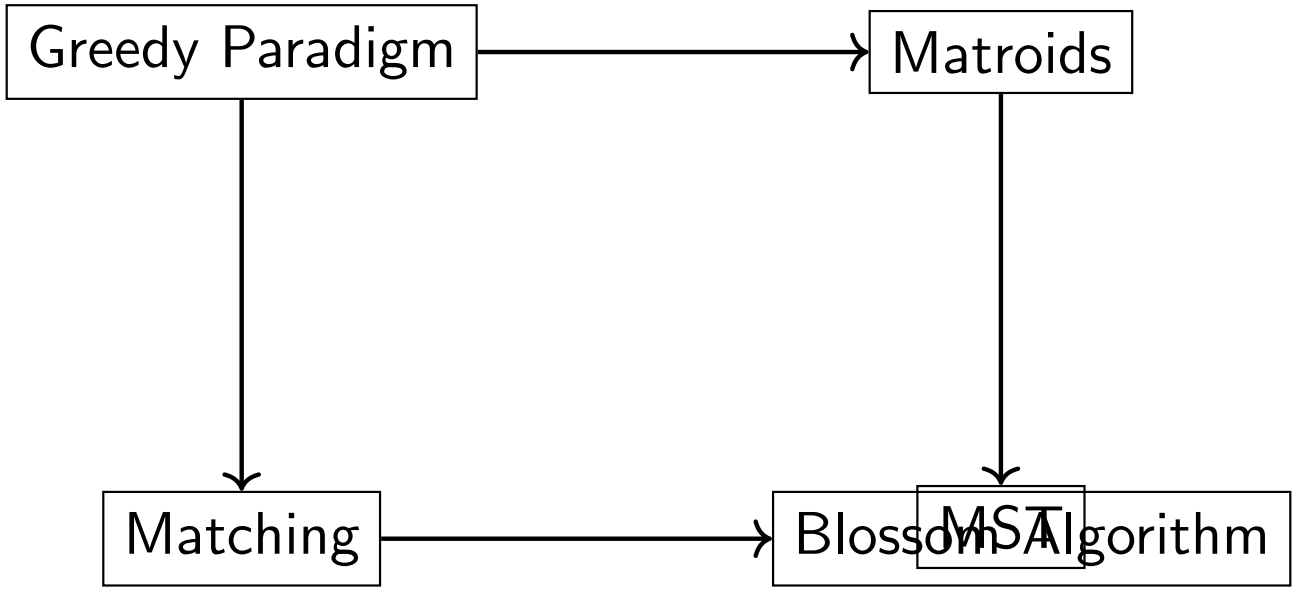
Blossoms: Why General Graphs Are Hard

- Odd cycles block simple augmenting paths.
- These cycles hide possible improvements.
- Such structures are called blossoms.
- They do not occur in bipartite graphs.
- Special handling is required.

Edmonds' Blossom Algorithm

- Detects odd cycles during search.
- Shrinks blossoms into super-vertices.
- Searches for augmenting paths in reduced graph.
- Expands blossoms carefully.
- Guarantees maximum matching.

Unit II: One Unified View (Visual)



Greedy vs Matroid vs Matching

- Greedy works when structure is simple.
- Matroids formalize that structure.
- MST fits perfectly into matroid framework.
- Matching breaks matroid assumptions.
- Augmenting paths and blossoms fix this.

Algorithm Design Lessons from Unit II

- Algorithm design depends on structure.
- Greedy is not universally applicable.
- When greedy fails, look for augmentation.
- Abstract theory simplifies correctness proofs.
- Advanced algorithms require multiple tools.

Exam and Viva Perspective

- Be clear about when greedy works and why.
- Always mention matroid properties explicitly.
- For matching, emphasize augmenting paths.
- Explain blossoms conceptually, not implementation.
- Focus on intuition and correctness arguments.

What Comes After Unit II

- Flow networks and max-flow algorithms.
- Approximation algorithms.
- Randomized algorithms.
- Advanced graph algorithms.
- Unit II prepares the foundation for all of these.

Final Summary of Unit II

- Greedy algorithms need structural guarantees.
- Matroids provide these guarantees.
- MST is a clean matroid application.
- Matching needs augmenting paths and blossoms.
- Unit II builds deep algorithmic thinking.